
Metodologia de Calibração e *Tracking* da Bola

Visão computacional para futebol de robôs VSSS

Ideia central. Um modelo robusto, porém caro, aprende com o vídeo e produz referências confiáveis. Depois, um modelo leve é escolhido para se aproximar dessas referências com baixa latência. Quadros difíceis são separados para revisão humana.

Projeto Robot Soccer Vision

Documento conceitual — setembro de 2026

Sumário

1	O problema em poucas palavras	1
2	Calibração inicial	1
2.1	Correção de perspectiva com um quadrilátero	1
2.2	Amostra inicial da bola	1
3	Representação de cor e adaptação à luz	2
4	Pré-processamento comparado	3
5	Técnicas de detecção	3
5.1	<i>Matched filter</i> disco-menos-anel	3
5.2	Componentes conexos	3
5.3	Transformada de Hough	4
6	O <i>teacher</i> robusto	4
7	<i>Self-supervision</i> e escolha de hiperparâmetros	4
7.1	Ajuste do <i>teacher</i>	5
7.2	Escolha do <i>student</i> por <i>ablations</i>	5
8	Modelo leve e controle de latência	5
8.1	Como e por que medir a latência	6
9	<i>Active learning</i> e revisão humana	6
10	Avaliação e interpretação correta	7
11	Resumo visual	7

1 O problema em poucas palavras

A câmera observa o campo por cima e permanece fixa. A bola é laranja, aproximadamente circular e quase não muda de tamanho na imagem. Essas três informações são muito úteis:

- a **cor** reduz a busca às regiões parecidas com laranja;
- a **forma circular** rejeita linhas, etiquetas e manchas;
- o **raio quase constante** elimina objetos grandes ou pequenos demais.

O desafio é que a aparência da cor muda com iluminação, exposição, sombras e reflexos. Além disso, o sistema de *tracking* precisa responder rapidamente. Na terminologia usual de *machine learning*, o sistema separa duas funções:

Teacher robusto	Usa várias técnicas e busca consenso. É mais lento e serve como <i>teacher</i> durante calibração e avaliação.
Student leve	Usa apenas a combinação escolhida pelos testes comparativos, chamados de ablações. É o modelo executado em tempo real durante o jogo.

O modelo robusto não é uma verdade perfeita. A posição que ele gera automaticamente é um *pseudo-label*, não um *ground truth*. Uma pequena amostra rotulada manualmente continua necessária para detectar erros que os dois modelos possam cometer juntos.

Exemplo para acompanhar o texto: suponha uma bola de raio 17 pixels. Em certo quadro, o *teacher* estima o centro em (620,410) e o *student* em (622,409). A distância entre os centros é 2,2 pixels, ou $2,2/17 = 0,13$ raio. Se o resultado chegar 3,8 ms após a captura, esse quadro contribui tanto para a medida de precisão quanto para a de latência.

2 Calibração inicial

2.1 Correção de perspectiva com um quadrilátero

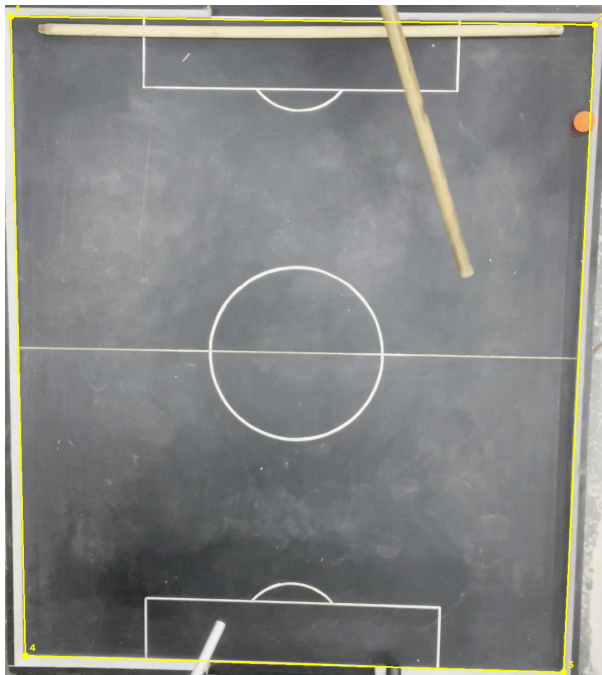
O usuário marca os quatro cantos do campo na ordem: superior esquerdo, superior direito, inferior direito e inferior esquerdo. Esses pontos definem um quadrilátero. Uma **homografia** transforma esse quadrilátero em um retângulo, removendo a inclinação aparente causada por uma câmera lateralmente deslocada ou não perfeitamente perpendicular.

O quadrilátero é usado **somente para corrigir o enviesamento da imagem**. Ele não converte pixels em milímetros e não inventa margens fora da região selecionada. O mapa da transformação é calculado uma vez e reutilizado nos demais quadros.

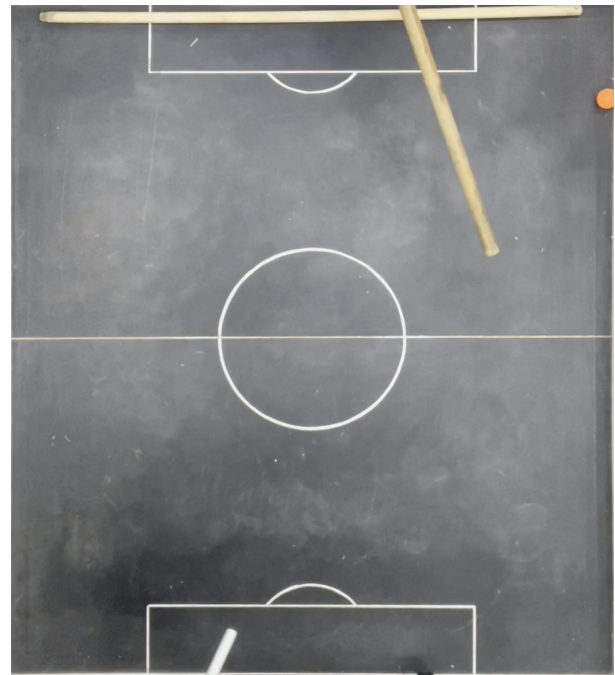
A Figura 1 mostra um exemplo obtido do primeiro quadro de um vídeo de treinamento. À esquerda, os quatro pontos escolhidos delimitam o campo. À direita, a mesma região aparece após a homografia: os cantos passam a formar um retângulo e as áreas externas ao quadrilátero são descartadas.

2.2 Amostra inicial da bola

Na imagem corrigida, o usuário posiciona uma circunferência sobre a bola e ajusta seu raio. Os pixels da parte interna dessa circunferência fornecem a primeira amostra de cor. Usa-se a mediana dos valores, porque ela é menos sensível a um pequeno reflexo branco ou a um pixel com ruído.

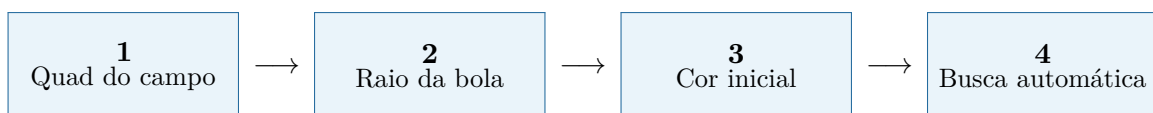


(a) Antes: quadrilátero selecionado.



(b) Depois: campo retificado.

Figura 1: Correção de perspectiva aplicada ao primeiro quadro do vídeo de treinamento.



3 Representação de cor e adaptação à luz

A imagem chega em BGR, a ordem azul-verde-vermelho usada pelo OpenCV, e é convertida para **HSV**, que reorganiza a mesma cor em matiz, saturação e brilho. Nesse espaço, cada pixel é descrito por:

- **H (matiz)**: o tipo de cor, por exemplo laranja;
- **S (saturação)**: quão viva ou acinzentada é a cor;
- **V (valor)**: o brilho.

Um pixel entra na máscara de bola quando sua distância para a cor aprendida está dentro das tolerâncias de H, S e V. A matiz é circular: valores próximos das extremidades da escala também podem representar cores vizinhas.

A **máscara binária** é uma imagem em preto e branco: um pixel aceito pelas três tolerâncias fica branco (possível bola), enquanto um pixel rejeitado fica preto. Ela evita cálculos com números decimais e pode ser processada rapidamente tanto pelo *teacher* quanto pelo *student*; o *teacher* apenas combina essa informação com mais evidências.

Depois de uma detecção de alta confiança, o sistema observa novamente a parte interna da bola e atualiza a cor por uma **média móvel exponencial**:

$$\text{cor nova} = (1 - \alpha) \text{cor anterior} + \alpha \text{cor observada}.$$

O fator α é a taxa de aprendizagem. Um valor pequeno acompanha mudanças graduais sem reagir demais a um único quadro. A atualização possui travas: exige boa forma circular, boa confiança, saturação mínima e limita a mudança de matiz por quadro. Se a bola não for encontrada, o modelo

de cor fica congelado para evitar que o fundo seja aprendido por engano.

4 Pré-processamento comparado

A etapa de ajuste testa quatro alternativas. Nenhuma é sempre melhor; a escolha depende do vídeo e do custo medido no computador real.

Alternativa	Intuição e principal compromisso
Sem filtro	Preserva a máscara original e tem custo mínimo, mas é mais sensível a pixels isolados.
Gaussiano 5×5	Suaviza variações locais antes da decisão, porém pode borrar a borda da bola.
Abertura circular	Erosão seguida de dilatação remove pequenos pontos, com risco de reduzir objetos muito pequenos.
Abertura fechamento	+ Além de remover pontos, preenche pequenos buracos causados por reflexo, ao custo de mais operações.

Os elementos morfológicos são elípticos, não quadrados. Isso respeita melhor a geometria da bola e evita introduzir preferência artificial por direções horizontal e vertical. A **erosão** encolhe as regiões brancas da máscara, enquanto uma **dilatação** as expande. A abertura combina as duas para remover pontos pequenos; o fechamento usa a ordem contrária para preencher pequenos buracos.

5 Técnicas de detecção

5.1 *Matched filter* disco-menos-anel

O detector leve principal usa uma **convolução** com um *kernel* circular pré-calculado. O *kernel* é uma pequena tabela de números que funciona como um molde. A convolução desliza esse molde sobre a imagem e calcula, em cada posição, uma nota de semelhança. O centro do *kernel* possui pesos positivos e representa a bola. Um anel externo possui pesos negativos e representa o fundo esperado ao redor dela.

Uma bola laranja isolada produz resposta alta: o interior combina com o disco positivo e o entorno não ativa o anel negativo. Uma grande região laranja tende a invadir o anel e perde pontuação.

Como o raio muda pouco, o *kernel* é criado uma vez. A convolução procura o ponto de maior resposta. Essa operação é densa, simples e rápida em uma pequena região de busca.

5.2 Componentes conexos

A máscara também pode ser dividida em regiões conectadas. Para cada região, calculam-se área, perímetro, círculo envolvente, circularidade e diferença para o raio esperado. Regiões com tamanho incorreto são descartadas. Essa abordagem é barata quando a máscara contém poucos objetos, mas pode sofrer quando muitas manchas têm a mesma cor.

5.3 Transformada de Hough

O modelo robusto usa ainda a transformada de Hough para círculos. Ela procura evidências de bordas organizadas ao redor de um centro e de um raio. É mais cara, mas fornece uma fonte de evidência diferente da cor preenchida e dos componentes conexos.

6 O *teacher* robusto

O *teacher* executa uma busca global e combina várias opiniões:

1. *matched filter* (filtro casado) e componentes conexos;
2. repete a detecção com três tamanhos candidatos próximos ao raio inicial, tolerando um pequeno erro da calibração;
3. abertura e fechamento morfológicos;
4. detecção circular por Hough;
5. preferência temporal por posições compatíveis com o movimento recente.

As propostas próximas são agrupadas. Cada grupo recebe votos e uma pontuação de acordo com confiança, concordância espacial e proximidade da posição prevista. Só grupos com consenso mínimo geram um *pseudo-label*. A cor adaptativa do *teacher* também só é atualizada após uma decisão confiável.

Por exemplo, se três detectores propõem centros separados por apenas dois ou três pixels, eles formam um grupo forte. Uma quarta proposta muito distante fica isolada e pode ser rejeitada. A posição aceita recebe o nome de *pseudo-label*: é um rótulo produzido automaticamente, útil para treinamento, mas ainda sujeito a erro.

Como trabalha fora de tempo real, o *teacher* analisa o vídeo nos dois sentidos. Detecções próximas feitas com passado e futuro são combinadas conforme a confiança; grandes diferenças são rejeitadas. Um *Rauch–Tung–Striebel smoother* bidirecional ajusta posição e velocidade na trajetória inteira. Lacunas curtas só são preenchidas quando existem posições compatíveis antes e depois, sem inventar dados durante oclusões longas.

Esse conjunto custa mais tempo que o permitido no jogo, mas reduz a chance de confiar em um único tipo de erro. Ele é adequado para processar vídeos gravados fora do laço de controle.

	Informação temporal	Uso principal
<i>Teacher</i>	Quadro atual, passado e futuro; <i>RTS smoother</i> bidirecional.	Criar e conferir <i>pseudo-labels</i> em vídeo gravado.
<i>Student</i>	Quadro atual e estado calculado no passado; filtro de Kalman causal.	Entregar a posição ao vivo durante o jogo.

Usar o futuro no *teacher* não é *data leakage* para o sistema ao vivo: essa informação serve somente para criar rótulos fora de tempo real. O *student* nunca recebe quadros futuros durante o ajuste nem durante o teste.

7 *Self-supervision* e escolha de hiperparâmetros

Self-supervision (autossupervisão) significa criar um sinal de qualidade a partir do próprio vídeo, sem exigir a posição medida por um sensor externo em todos os quadros.

7.1 Ajuste do *teacher*

O sistema experimenta variações da cor inicial, raio, tolerâncias HSV, tolerância de raio e taxa de adaptação. Cada candidato é avaliado por quatro sinais:

- **cobertura:** quantos quadros recebem uma detecção;
- **confiança:** quão forte é o consenso interno do *teacher*;
- **ida e volta:** o vídeo processado do início ao fim deve concordar com o mesmo vídeo processado do fim ao início;
- **iluminação:** a posição deve permanecer semelhante após alterações artificiais de brilho e saturação.

O melhor *teacher* produz *pseudo-labels* bidirecionais.

7.2 Escolha do *student* por *ablations*

O *student* permanece causal: usa somente o quadro atual e resultados anteriores, nunca o futuro. Comparam-se então 16 combinações: quatro pré-processamentos, dois detectores e busca por região ligada ou desligada. Uma ***ablation*** é esse teste controlado: muda-se uma parte do *pipeline* e observa-se quanto a precisão e a latência se alteram. O critério final combina:

$$\text{objetivo} = \text{erro de centro normalizado} + \text{penalidade por falhas} + \text{penalidade por latência}.$$

Esse procedimento seleciona um modelo que não é apenas rápido, nem apenas parecido com o *teacher*: ele procura um equilíbrio entre ambos.

Antes da soma, cada termo é colocado em escala comparável e recebe um peso definido no experimento. O erro de centro normalizado é a distância entre os centros do *student* e do *teacher* dividida pelo raio esperado da bola. Assim, no exemplo inicial, 2,2 pixels viram 0,13 raio. As penalidades por falha e latência impedem que uma solução lenta ou que perde a bola pareça boa apenas por acertar os poucos quadros restantes.

8 Modelo leve e controle de latência

Durante o jogo, o modelo leve usa a velocidade recente para prever onde a bola estará e recorta uma **região de interesse** (ROI), isto é, um pequeno retângulo provável em vez da imagem inteira. Cor, pré-processamento e detecção são executados apenas nessa região. Se a bola for perdida, o sistema volta temporariamente para uma busca global em meia resolução.

Um filtro de Kalman **causal** mantém uma estimativa de posição e velocidade usando apenas medições que já chegaram. Ele prevê o próximo centro, e a nova detecção corrige essa previsão. Isso define a ROI com custo muito pequeno. É diferente do *RTS smoother* do *teacher*, que revisa a trajetória depois de conhecer também os quadros futuros.

Outras medidas reduzem atraso:

- o *kernel* circular é pré-calculado;
- a captura mantém somente o quadro mais novo e descarta quadros antigos;
- o mapa de correção de perspectiva é pré-calculado;
- a interface informa tempo de processamento, latência captura-até-resultado e uso de ROI;
- a cor só se adapta depois de detecções confiáveis.

8.1 Como e por que medir a latência

Uma linha de execução separada, chamada *thread* de captura, recebe cada quadro e registra um instante $t_{captura}$ usando um relógio monotônico. Logo depois da correção de perspectiva e da localização da bola, mas antes de desenhar a interface, registra-se $t_{resultado}$. A medida principal é:

$$L_{captura \rightarrow resultado} = t_{resultado} - t_{captura}.$$

Ela representa aproximadamente há quanto tempo os dados usados para calcular a posição já estavam no computador. A renderização fica fora da medida porque não é necessária para o robô tomar uma decisão e pode ser desativada em produção. A correção de perspectiva fica dentro, pois é indispensável para obter a posição usada pelo controle.

Também se registra o **tempo de processamento**, medido apenas da entrada no pipeline à saída do detector. Ele ajuda a otimizar o algoritmo, mas não substitui a latência completa. Um programa pode processar cada quadro em 2 ms e, ainda assim, entregar uma imagem antiga se ela passou 20 ms esperando em uma fila. Da mesma forma, muitos quadros por segundo medem *throughput*, não garantem que o resultado mais recente chegou cedo. Por isso a captura usa um único slot e substitui quadros antigos pelo mais novo.

O relatório deve apresentar pelo menos média, mediana, percentil 95 e máximo. A média mostra o comportamento comum; o percentil 95 revela atrasos frequentes que a média esconde; o máximo expõe travamentos ocasionais. Nos testes, descartam-se alguns quadros iniciais de aquecimento e mede-se por tempo suficiente para incluir buscas por ROI e recuperações globais.

Sem timestamps fornecidos pela própria câmera, $t_{captura}$ é o instante em que o driver entrega o quadro ao programa. Portanto, essa medida não inclui com precisão exposição e transferência interna da câmera; ela é um limite inferior da latência física total. Quando o SDK da câmera fornecer timestamps de hardware, eles devem substituir esse instante para uma medição ponta a ponta mais fiel.

Uma analogia simples é iniciar um cronômetro depois que parte da corrida já aconteceu: o tempo registrado pode ser útil para comparar algoritmos, mas o percurso físico completo só pode ter demorado mais.

9 *Active learning* e revisão humana

Rotular todos os quadros é caro e pouco informativo, pois a maioria deles é fácil. O *active learning* calcula uma dificuldade para cada quadro usando:

- baixa confiança e poucos votos do *teacher*;
- diferença entre *teacher* e *student*;
- movimento inesperado em relação à previsão temporal;
- pouca nitidez, medida pela variância do Laplaciano;
- baixo brilho e grande fração de sombras.

Forma-se um conjunto com os quadros mais difíceis — por padrão, os 25% de maior pontuação — e sorteiam-se N exemplos uniformemente desse conjunto. O sorteio evita revisar apenas um trecho contínuo do vídeo. O usuário pode marcar o centro e o raio ou pular quadros impossíveis por desfoque de movimento, escuridão ou oclusão. O motivo do descarte também é guardado e pode revelar limitações da câmera ou da iluminação. Pular significa “não é possível rotular este quadro”, e não “a bola está ausente”.

10 Avaliação e interpretação correta

Na avaliação, *teacher* e *student* recebem exatamente o mesmo quadro corrigido. São registrados erro entre centros, falhas do *student*, detecções exclusivas do *student*, confiança, latência e quadros descartados pela captura. Os resultados agregados incluem média, mediana, percentil 95 e erro máximo, além de concordância relativa ao raio da bola.

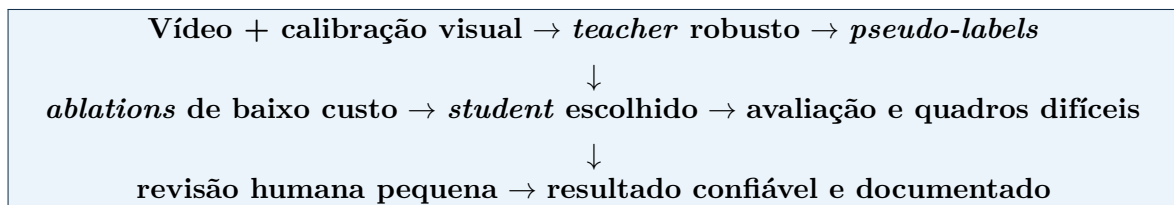
Essas medidas respondem a três perguntas diferentes:

- **O *student* imita o *teacher*?** Compare seus centros e falhas no mesmo vídeo.
- **Os dois estão corretos?** Compare ambos com a amostra manual independente.
- **O *student* atende ao tempo real?** Meça a latência no computador e na câmera usados no jogo.

Como interpretar: boa concordância com o *teacher* mostra que o *student* preservou o comportamento do modelo robusto. Ela não prova que ambos estão corretos. A validação final deve comparar os dois com uma pequena amostra manual independente.

Para permitir um relatório científico posterior, cada etapa registra caminhos, hashes SHA-256, parâmetros, versões das bibliotecas, revisão Git, resultados por quadro e artefatos gerados. Assim, tabelas e gráficos podem ser reproduzidos sem depender de anotações feitas à mão durante o experimento.

11 Resumo visual



Este documento descreve os princípios da metodologia implementada no projeto Robot Soccer Vision. Ele prioriza intuição e interpretação, não detalhes de código.